

บทที่ ๑๐

ตรรกะลำดับหนึ่ง



Lecture 10 - First-order Logic

CS265 Artificial Intelligence Fundamentals

Thapana Boonchoo, Ph.D.
Department of Computer Science
Thammasat University
thapana@cs.tu.ac.th

In which we notice that the world is blessed with many objects, some of which are related to other objects, and in which we endeavor to reason about them.

First-order Logic

- ใน Lecture ก่อนหน้าเราได้พิจารณาถึง Knowledge-based agent ที่สามารถ Represent โลกที่มันอยู่และสามารถอนุมานได้ว่าควรต้องแสดงการกระทำใด
- เราได้มีการใช้ Propositional logic เพื่อเป็น Representation language เพราะสามารถที่จะแสดงได้ถึงแนวคิดพื้นฐานสำหรับ Logic และ Knowledge-based agent
 - แต่ Propositional logic นี้เป็นภาษาที่ค่อนข้างอ่อนแอ (Puny) ที่ใช้ในการแทนองค์ความรู้ (Knowledge) ของสภาพแวดล้อมที่ซับซ้อนได้อย่างกะทัดรัด (Concise)
- ใน Lecture นี้เราจะพิจารณา **First-order logic** ซึ่งสามารถที่จะแทนความรู้ที่เป็น Commonsense ได้
- First-order logic ยังเป็น Foundation ของ Representation languages อื่นๆ และมีการศึกษามาแล้วอย่างยาวนาน

Representation Revisited

Representation Revisited

- ภาษาโปรแกรม เช่น C++, Java ถือเป็น Formal languages ที่ตัวโปรแกรมนั้นเป็นการแทนขั้นตอนการประมวลผล (Computational process)
 - เช่น โปรแกรมอาจจะใช้ 4x4 Array ในการแทนโลก Wumpus world และ `World[2,2] <- Pit` จึงหมายถึงการที่บ่งบอกถึงการมีอยู่ของหลุมในช่อง [2,2]
- สิ่งที่ภาษาโปรแกรมขาดคือ Mechanism ที่จะ Derive Facts จาก Facts อื่น ๆ
- การอัปเดต (เปลี่ยนแปลงค่า) Data structure ในภาษาโปรแกรมจึงถูกทำโดยความรู้ของโปรแกรมเมอร์ใน Field นั้น ๆ
 - กระบวนการลักษณะนี้ตรงกันข้ามกับ “Declarative” ของ Propositional logic
 - Declarative คือ Knowledge และ Inference นั้นถูกแยกส่วนออกจากกัน และ Inference เป็น Domain-independent
- Propositional logic ถือเป็น Declarative language เพราะว่า Semantics นั้นขึ้นกับความสัมพันธ์เชิงความจริง (Truth relation) ระหว่าง Sentence กับ Possible worlds
- Propositional logic ก็มีความสามารถมากพอที่จะจัดการกับข้อมูลบางส่วน (Partial information) ด้วย Disjunction และ Negation

Representation Revisited

- Propositional logic ยังมีอีกคุณสมบัติหนึ่งคือ Compositionality กล่าวคือ ความหมายของประโยคหนึ่ง เป็นฟังก์ชันของความหมายของแต่ละส่วนในประโยค
 - เช่น ความหมายของ $S_{1,4} \wedge S_{1,2}$ เกี่ยวโยงกับความหมายของทั้ง $S_{1,4}$ และ $S_{1,2}$
- แต่ Propositional logic ยังขาดความสามารถในการอธิบายสภาพแวดล้อมที่มีหลายวัตถุได้อย่างกระชับ (Concisely) กล่าวคือ เราถูกบังคับให้เขียน Rule แยกกันเกี่ยวกับ Breezes และ Pits
 - เช่น เราจะต้องเขียนกฎที่แยกจากกันเกี่ยวกับ Breezes และ Pits สำหรับแต่ละช่อง $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- ในภาษามนุษย์ มันง่ายที่จะพูดว่า “ช่องที่ติดกับหลุมนั้นจะมี Breezes” และอาจจะทำให้การอธิบาย Environment นี้ได้กระชับขึ้น (แต่จากตัวอย่างด้านบน เราต้องเขียน Rule สำหรับทุกช่อง)

Representation Revisited

- ตลอดบทเรียนที่ผ่านมา เรามีการพูดคุยกันโดยใช้ภาษาธรรมชาติ (ภาษาอังกฤษ/ไทย) และมีการสลับไปพูดถึงภาษาอื่น ๆ บ้างเช่น (ภาษา) คณิตศาสตร์ ตรรกะ Diagrams
- หากเราสามารถที่จะอธิบายกฎต่าง ๆ สำหรับ ภาษามนุษย์ (ภาษาธรรมชาติ – Natural language) เราอาจจะใช้มันในระบบการแทนองค์ความรู้และให้เหตุผล (Representation and reasoning systems) และอาจจะได้ประโยชน์จากหนังสือหลายล้านเล่ม
- แต่ขณะนี้เรามองภาษามนุษย์เป็นสื่อกลางการสื่อสาร มากกว่าการใช้แทนองค์ความรู้
 - ภาษามนุษย์นั้นมีข้อจำกัดเรื่อง **ความกำกวม (Ambiguity)** ซึ่งเป็นปัญหาสำหรับการแทนองค์ความรู้
- สำหรับ First-order logic reasoning system ที่มีการใช้ CNF (Chomsky normal form) กล่าวคือเราจะพบว่ารูปภาษา “ $\neg(A \vee B)$ ” และ “ $\neg A \wedge \neg B$ ” นั้นคือสิ่งเดียวกันจากการที่เราเห็นว่าทั้งสองประโยคนี้สามารถเขียนในรูป CNF รูปเดียวกันได้
- จากมุมมองของ Formal logic การแทนองค์ความรู้ที่เหมือนกันด้วยวิธีที่ต่างกัน ไม่ทำให้เกิดความแตกต่างเลย กล่าวคือ Facts เดียวกันสามารถ Derive กลับไป-มา ได้

Combining the best of formal
and natural languages

⁹Combining the best of formal and natural languages

- จากทั้ง Propositional logic ที่เป็น Declarative and compositional semantics (Context-independent และ ไม่กำกวม) เราสามารถสร้าง Logic ที่มีการอธิบายได้ดีขึ้น (More expressive) โดยยืมแนวคิดการ Representation จากภาษามนุษย์โดยระมัดระวังข้อด้อยของมัน
- องค์ประกอบหลักในภาษามนุษย์
 - **Objects** : Noun หรือ Noun phrase เช่น Squares, Pits, Wumpuses, people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries . . .
 - **Relations**: Verb หรือ Verb phrase ระหว่าง Objects เช่น Breezy, is adjacent to, shoots อาจจะเป็น Unary (Properties) เช่น red, round, bogus, prime, multistoried หรืออาจจะเป็น n-ary relations เช่น brother of, bigger than, inside, part of, has color, occurred after, owns, comes between, . . .
 - **Functions**: Relations ที่มีเพียง 1 ค่าสำหรับ Input 1 ค่า เช่น father of, best friend, third inning of, one more than, beginning of . . .

Combining the best of formal and natural languages

- ตัวอย่าง 1: “One plus two equals three”
 - **Object**: One, Two, Three, One plus two
 - **Relation**: Equals
 - **Function**: Plus
 - “One plus Two” เป็นชื่อ Object ที่เกิดจากการ Apply function Plus ไปยัง Objects One และ Two โดยที่ Three ก็เป็นอีกชื่อหนึ่งของ Object นี้
- ตัวอย่าง 2: “Squares neighboring the wumpus are smelly.”
 - **Object**: Wumpus และ Squares
 - **Relation**: Neighboring
 - **Property** (Unary relation): Smelly

Combining the best of formal and natural languages

- ภาษา First-order logic นั้นถูกสร้างขึ้นจาก Objects และ Relations
- ซึ่ง First-order logic มีความจำเป็นมากสำหรับวิชาหลาย ๆ แขนงที่เกี่ยวข้องกับการมีชีวิตทุก ๆ วันของมนุษย์ที่จะต้องเกี่ยวข้องกับ Objects และ Relations ตลอดเวลา
- First-order logic สามารถบ่งบอกถึง Facts เกี่ยวข้องกับ บางส่วน (Some) หรือ ทั้งหมด (All) ของ Objects ใน Universe ได้
- First-order logic จึงทำให้เราสามารถ Represent กฎทั่วไป (General laws/rules) เช่นประโยคที่บอกว่า “Squares neighboring the wumpus are smelly.” ได้

Combining¹² the best of formal and natural languages

- ความแตกต่างหลัก ๆ ระหว่าง Propositional logic และ First-order logic คือ Ontological commitment (คือสิ่งที่เป็นสมมติฐานของธรรมชาติของความจริง)
 - Ontological commitment นี้แสดงได้จากธรรมชาติของ Formal models ที่ค่าความจริงของประโยคนั้นถูกนิยาม เช่น
 - **Propositional logic** มีสมมติฐานว่า Facts นั้นจะต้องมีค่าความจริงไม่ True (Hold) ก็ False (Not hold) เท่านั้นในโลก
 - **First-order logic** มีสมมติฐานสูงขึ้นไปกว่านั้นกล่าวคือ โลกของเราประกอบไปด้วย Objects ที่มีความสัมพันธ์ (Relations) ระหว่างกันที่อาจจะ Hold หรือ Not hold ก็ได้ ซึ่งซับซ้อนกว่า Propositional logic
 - **Temporal logic** มีสมมติฐานว่า Facts จะ Hold และ Not hold บางช่วงเวลา
 - **Higher-order logic** จะมองว่า Relations และ Functions เป็น Objects
 - Epistemological commitments: Logic สามารถแบ่งได้ตาม Possible states ของ Knowledge ที่เป็นไปได้สำหรับ Facts
 - Propositional logic และ First-order logic นั้นมีประโยคที่จะเป็น จริง หรือ เท็จ หรือ ไม่มีความเห็น
 - ในขณะที่ระบบที่ใช้ Probability theory นั้นจะมี Degree of believe (0 ถึง 1 – disbelieve -> total believe)

Combining the best of formal and natural languages

Language	Ontological Commitment (What exists in the world)	Epistemological Commitment (What an agent believes about facts)
Propositional logic	facts	true/false/unknown
First-order logic	facts, objects, relations	true/false/unknown
Temporal logic	facts, objects, relations, times	true/false/unknown
Probability theory	facts	degree of belief $\in [0, 1]$
Fuzzy logic	facts with degree of truth $\in [0, 1]$	known interval value

Formal languages and their ontological and epistemological commitments.

Syntax and Semantics of First-order Logic

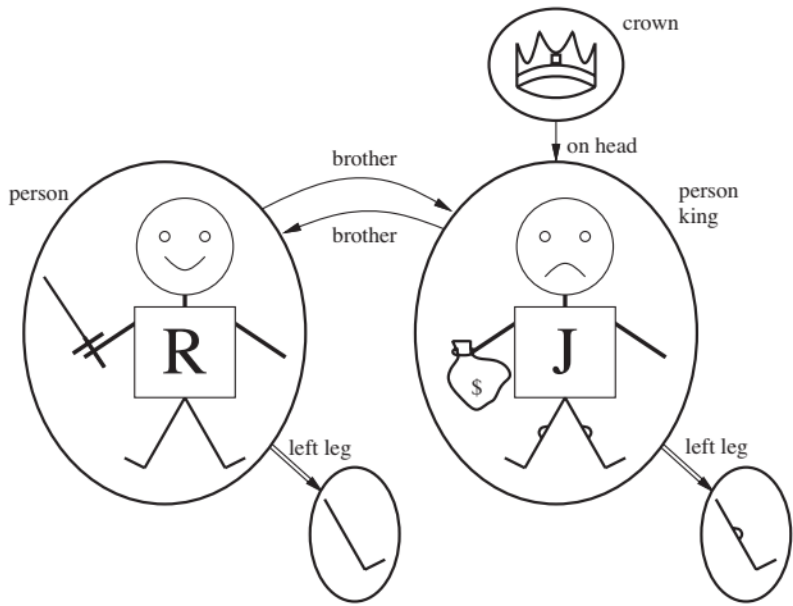
Models for first-order logic

Models for first-order logic

- จาก Models ใน Logical language (Propositional logic) ในบทก่อนหน้านี้คือเรากำลังพิจารณา Formal Structure ทุก ๆ Worlds ที่เป็นไปได้
 - แต่ละ Model นั้นเชื่อมโยง Vocabulary ใน Logical Sentences ไปยัง Possible worlds และค่าความจริงของ Sentence จึงถูกระบุได้ กล่าวคือ Model ใน Propositional logic เชื่อมโยง Propositional symbols ไปยังค่าความจริงในแบบต่าง ๆ
- Models ใน First-order logic นั้นน่าสนใจกว่าที่ว่ามันมี Objects โดยที่ Domain ของ Model หนึ่งนั้นคือ **Set of objects** โดยทุก ๆ Domain นั้นจะต้องไม่เป็นเซตว่าง (ต้องมีอย่างน้อยที่สุด 1 Object)
 - ไม่สำคัญว่า Objects เหล่านี้คืออะไร แต่สำคัญว่ามันมีกี่ Objects ในโมเดลหนึ่ง ๆ
- โดยที่ Relation คือ **Set of tuples of objects** ที่เกี่ยวข้องกัน

Tuple คือ collection of objects ที่มีลำดับที่ชัดเจนและถูกเขียนอยู่ใน angle brackets

17 Models for first-order logic



ความสัมพันธ์เชิงพี-น้องในโมเดลนี้สามารถเขียนแทนได้ดังนี้
{ <Richard the Lionheart, King John>, <King John, Richard the Lionheart> }

Function LeftLeg
<Richard the Lionheart> → Richard's left leg
<King John> → John's left leg .

- ในรูปนี้มีทั้งหมด 5 Objects 2 Binary relations และ 3 Unary relations
- 5 Objects ได้แก่
 - Richard the Lionheart (King of England (1189-1199))
 - The evil King John (his younger brother) (1199-1215)
 - Left legs ของทั้ง 2 คนด้านบน
 - แล้วยัง Crown (มงกุฎ)
- Relations (Binary):
 - Richard และ John นั้นเป็นพี่น้องกัน
 - On head: เขียน โดยใช้เพียง 1 Tuple คือ {<the crown, King John>}
- Relation (Unary):
 - Person (Object เป็นคน?): พบว่า Person จะ True สำหรับทั้ง Richard และ John
 - King (Object เป็น King?): พบว่า King จะ True สำหรับ John เท่านั้น

Symbols and interpretations

Symbols and interpretations¹⁹

- สำหรับไวยากรณ์ของ First-order logic คือ สัญลักษณ์(Symbols) สำหรับ Objects, Relations และ ฟังก์ชัน
 - **Constant symbols** สำหรับ Objects; เช่น *Richard, John, ...*
 - **Predicate symbols** สำหรับ Relations; เช่น *Brother, OnHead, Person, King*
 - **Function symbols** สำหรับ Functions; เช่น *LeftLeg*
 - **Convention:** Symbols เหล่านี้จะขึ้นต้นด้วยตัวพิมพ์ใหญ่
- ใน Propositional logic ทุก Model จะต้องมีการระบุ Information ที่จะบ่งบอกได้ว่า Sentence นั้นจะมีค่าความจริง True หรือ False
 - ดังนั้นใน Model นั้นนอกจาก Objects, Relation และ Functions แล้วแต่ละ Model ต้องมี **Interpretation** ด้วยเพื่อระบุว่า Objects, Relation และ Functions อ้างถึงสัญลักษณ์อะไรอยู่ (Intended Interpretation) เช่น
 - *Richard* อ้างถึง Richard the Lionheart
 - *Brother* อ้างถึง ความสัมพันธ์เชิงพี่น้อง Set of tuples
 - *LeftLeg* อ้างถึง Leftleg function
 - อาจจะมี Interpretation อื่น ๆ ได้ เช่น *Richard* อ้างถึง Crown { <Richard the Lionheart, King John>, <King John, Richard the Lionheart> }
 - ใน Model มีทั้งหมด 5 Objects ดังนั้นจะมี 25 Interpretation สำหรับ Objects John และ Richard (5 x 5)
 - ไม่ใช่ทุก Object จำเป็นจะต้องมีชื่อ เช่น the crown หรือ the legs
 - หรือบาง Objects อาจจะมีหลายชื่อ

²⁰Symbols and interpretations

Sentence $\rightarrow$ *AtomicSentence* | *ComplexSentence*
AtomicSentence $\rightarrow$ *Predicate* | *Predicate*(*Term*,...) | *Term* = *Term*
ComplexSentence $\rightarrow$ (*Sentence*) | [*Sentence*]
| $\neg$ *Sentence*
| *Sentence* $\wedge$ *Sentence*
| *Sentence* $\vee$ *Sentence*
| *Sentence* $\Rightarrow$ *Sentence*
| *Sentence* $\Leftrightarrow$ *Sentence*
| *Quantifier* *Variable*,... *Sentence*

Term $\rightarrow$ *Function*(*Term*,...)
| *Constant*
| *Variable*

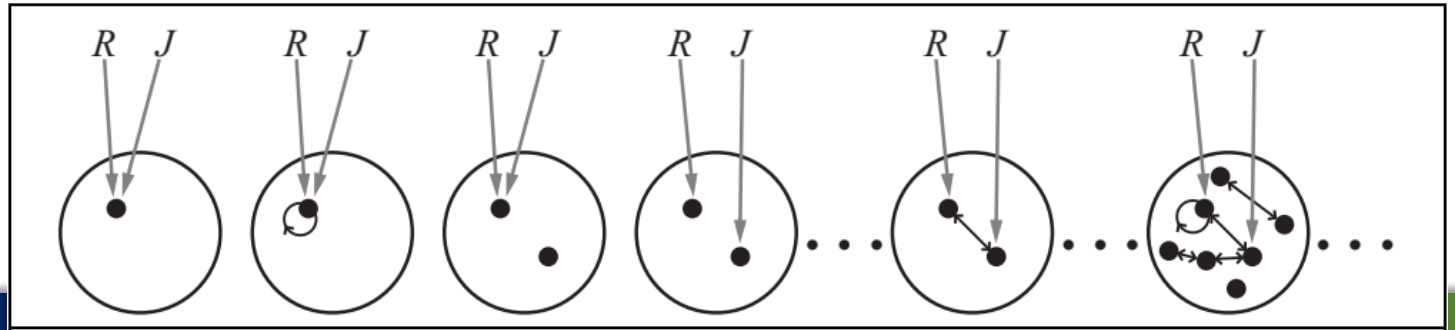
Quantifier $\rightarrow$ $\forall$ | $\exists$
Constant $\rightarrow$ *A* | *X*₁ | *John* | ...
Variable $\rightarrow$ *a* | *x* | *s* | ...
Predicate $\rightarrow$ *True* | *False* | *After* | *Loves* | *Raining* | ...
Function $\rightarrow$ *Mother* | *LeftLeg* | ...

OPERATOR PRECEDENCE : $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

The syntax of first-order logic with equality, specified in Backus–Naur form

Symbols and interpretations²¹

- โดยสรุป Model ใน First-order logic ประกอบไปด้วย Set ของ Objects และ Interpretation ที่ Map Constant symbols ไปยัง Objects, Predicate symbols ไปยัง relations และ Function symbols ไปยัง functions
- คล้ายกับ Propositional logic โดย Entailment, Validity และอื่น ๆ ถูกนิยามด้วย Models ที่เป็นไปได้ทั้งหมด (All possible models)
- พิจารณาจากรูปด้านล่างอธิบายถึง “All possible models”
 - การแสดงถึง Models นั้นแตกต่างกันไปตามจำนวน Objects (One up to infinity) และต่างกันที่ Constant symbols Map ไปยัง Objects ได้อย่างไรบ้าง
 - เมื่อมี 2 Constant symbols และมี 1 Object (Symbol นั้นอาจจะอ้างถึง Object เดียวกันได้)
 - เมื่อมี Objects มากกว่า Constant symbols อาจจะมีบาง Objects ที่ไม่มีชื่อได้
- จะเห็นว่าจำนวนของ Possible model ใน First-order logic นี้ไม่มีขอบเขต การตรวจสอบ Entailment ด้วยวิธีการ Enumeration จึงทำไม่ได้



- **Term** คือ Logical expression ที่อ้างถึง Object ดังนั้น Constant symbols จึงหมายถึง Terms
- เราไม่ได้จะให้ทุก ๆ Symbol มีชื่อของตัวเอง เช่น เราไม่ได้ให้ชื่อ Left leg ของ John แต่เราใช้ “King John’s left leg” และแทนที่เราจะใช้ Symbol เราใช้ Function แทน
 - เช่น “King John’s left leg” = $LeftLeg(John)$

Atomic/Complex sentences²³

- **Atomic sentences** ถูกสร้างจาก Predicate symbol (Symbols for relations) และตามด้วย List ของ Terms เช่น $Brother(Richard, John)$ หมายความว่า “Richard the Lionheart เป็นน้องชายของ King John”
 - Atomic sentences อาจจะมีรูปที่ซับซ้อนขึ้นอีกเช่น $Married(Father(Richard), Mother(John))$
 - Atomic sentence จะ **True** ใน Model เมื่อความสัมพันธ์ที่อ้างถึงด้วย Predicate symbol สอดคล้องกับ Objects อื่น ๆ ที่อ้างถึงใน Arguments
- **Complex sentences** เราใช้ logical connectives ในการสร้างประโยคความซ้อน เหมือนกับ Propositional logic
 - $\neg Brother(LeftLeg(Richard), John)$
 - $Brother(Richard, John) \wedge Brother(John, Richard)$
 - $King(Richard) \vee King(John)$
 - $\neg King(Richard) \Rightarrow King(John) .$

- ใน First-order logic เรามี Objects เราจะแสดง **คุณลักษณะ** ของ Objects มากกว่าการ Enumerate ทุก ๆ ชื่อ (Name) ที่เป็นไปได้ โดยใช้

Quantifiers

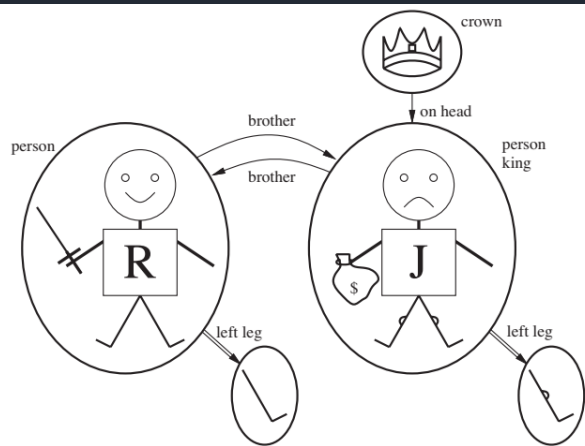
- ใน First-order logic มี Quantifiers 2 ชนิด
 - Universal quantifier ($\forall$)
 - Existential quantifier ($\exists$)

Universal quantifier ($\forall$)

Universal quantifier ($\forall$)

- จาก Propositional logic เราอาจจะมีกฎอย่างเช่น “Squares neighboring the wumpus are smelly” และ “All kings are persons” ซึ่งกฎลักษณะดังกล่าวเป็นใจความสำคัญของ First-order logic
- $\forall$ = For all ...
- “All kings are persons” สามารถเขียนในรูปของ First-order logic ได้ว่า
 - $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$
 - For all x, if x is a king, then x is a person
 - x ในที่นี้คือ ตัวแปร (Variable) หนึ่ง
 - ตัวแปรจะถูกเขียนด้วยตัวอักษรตัวพิมพ์เล็ก
- $\forall x P$ โดย P คือ Logical expression ที่เป็นจริงใน Model เมื่อ P เป็นจริงใน all Extended possible interpretations โดยที่แต่ละ Extended interpretation คือสมาชิกใด ๆ ที่ x อ้างถึง

Universal quantifier ($\forall$) (ต่อ)



Extended interpretations

$x \rightarrow$ Richard the Lionheart,

$x \rightarrow$ King John,

$x \rightarrow$ Richard's left leg,

$x \rightarrow$ John's left leg,

$x \rightarrow$ the crown.

$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ ประโยคนี้เป็นจริงเมื่อใน Model เมื่อ $\text{King}(x) \Rightarrow \text{Person}(x)$ เป็นจริงสำหรับทุกๆ Extended interpretations

ประโยค $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ จึงมีค่าเท่ากับ:

Richard the Lionheart is a king $\Rightarrow$ Richard the Lionheart is a person.

King John is a king $\Rightarrow$ King John is a person.

Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person. ?

John's left leg is a king $\Rightarrow$ John's left leg is a person. ?

The crown is a king $\Rightarrow$ the crown is a person. ?

บางประโยคกำลังสร้าง Claim เกี่ยวกับ Crown

หรือ Legs นั้นก็จะเป็นจริงไปด้วยแต่มันก็จะไม่

Qualified อยู่ดีเพราะ object เหล่านี้ไม่มีใครเป็น King

* นี่คือกฎของ $\Rightarrow$ กล่าวว่าเมื่อ premise (ข้อความด้านซ้าย) เป็นเท็จ ทั้งประโยคเป็นจริงไปโดยปริยาย

Universal quantifier ($\forall$) (ต่อ)

ประโยค $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ จึงมีค่าเท่ากับ:

Richard the Lionheart is a king $\Rightarrow$ Richard the Lionheart is a person.

King John is a king $\Rightarrow$ King John is a person.

Richard's left leg is a king $\Rightarrow$ Richard's left leg is a person.

John's left leg is a king $\Rightarrow$ John's left leg is a person.

The crown is a king $\Rightarrow$ the crown is a person.

จะเกิดอะไรขึ้นเมื่อเราเปลี่ยน **Conjunctive** $\Rightarrow$ เป็น $\wedge$

$\forall x \text{ King}(x) \wedge \text{Person}(x)$

Richard the Lionheart is a king $\wedge$ Richard the Lionheart is a person,

King John is a king $\wedge$ King John is a person,

Richard's left leg is a king $\wedge$ Richard's left leg is a person,

∴ **Assertions เหล่านี้จึงไม่ได้ capture ในสิ่งที่เราต้องการ**

Existential quantification ($\exists$)

Existential quantification ($\exists$)

- ในขณะที่ Universal quantification ($\forall$) สร้างประโยคเกี่ยวกับ ทุก ๆ Objects, **Existential quantification ($\exists$)** นั้นสร้างประโยคเกี่ยวกับบาง Objects เพื่อที่พูดว่า “King John has a crown on his head” เราใช้ $\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$
- $\exists x$ อ่านว่า “There exists an x such that ...” หรือ “For some x”
- ประโยค $\exists x P$ พูดว่า P เป็นจริงใน Model เมื่อ P เป็นจริงสำหรับ **อย่างน้อย** 1 Object x เช่น หนึ่งในประโยคต่อไปนี้เป็นจริง
 - Richard the Lionheart is a crown $\wedge$ Richard the Lionheart is on John's head;
 - King John is a crown $\wedge$ King John is on John's head;
 - Richard's left leg is a crown $\wedge$ Richard's left leg is on John's head;
 - John's left leg is a crown $\wedge$ John's left leg is on John's head;
 - The crown is a crown $\wedge$ the crown is on John's head.
- พบว่าประโยคที่ 5 เป็นจริงดังนั้น Original statement $\exists x P$ จึงเป็นจริงด้วย
- จะเห็นว่าการเขียนลักษณะที่มีการใช้ $\exists x$ ตรงกับความประสงค์ที่ว่า “King John has a crown on his head”

Existential quantification ($\exists$)

- ในขณะที่ $\Rightarrow$ มักถูกใช้สำหรับ $\forall$, ตัวเชื่อม $\wedge$ มักถูกใช้สำหรับ $\exists$
- การใช้ $\wedge$ ใน $\forall$ จะเป็นการสร้างประโยคที่ **Strong** เกินไป ในขณะที่ใช้ $\Rightarrow$ ใน $\exists$ จะเป็นการสร้างประโยคที่ **Weak** เกินไป
 - เช่น $\exists x \text{ Crown}(x) \Rightarrow \text{OnHead}(x, \text{John})$ ประโยคนี้จะเป็นจริงเมื่ออย่างน้อย 1 ในประโยคด้านล่างเป็นจริง

Richard the Lionheart is a crown $\Rightarrow$ Richard the Lionheart is on John's head;
 King John is a crown $\Rightarrow$ King John is on John's head;
 Richard's left leg is a crown $\Rightarrow$ Richard's left leg is on John's head;
 .
 .
 - จาก $\Rightarrow$ พบว่าประโยคจะเป็นจริงเมื่อ ประโยคทั้งซ้ายและขวาเป็นจริง หรือ Premise เป็นเท็จ ได้ว่า ถ้า Richard the Lionheart is not a crown ดังนั้น $\exists x \text{ Crown}(x) \Rightarrow \text{OnHead}(x, \text{John})$ เป็นจริงทันที (Too weak)
 - และเพราะ $\Rightarrow$ เป็นจริงในทุก ๆ กรณีที่ Premise ทุกตัวเป็นเท็จด้วย (ในกรณีนี้จึงแทบไม่ได้ให้ข้อมูลอะไรเลย)

Nested quantifiers³²

- เราสามารถแสดงประโยคที่มีความซับซ้อนมากขึ้นไปอีกโดยใช้ Nested quantifiers
- แบบแรกเป็นแบบที่ Quantifiers มีชนิดเดียวกัน เช่น “Brothers are siblings”
 - $\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$
 - สามารถเขียนด้วยตัว Quantifier ตัวเดียวหลายตัวแปรได้ $\forall x, y \text{ Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x)$
- กรณีที่มีการผสม เช่น
 - “Everybody loves somebody” (ความหมายถึงทุก ๆ มีคนที่รักบางคน) เขียนได้ว่า $\forall x \exists y \text{ Loves}(x, y)$.
 - “There is someone who is loved by everyone” (มีบางคนถูกรักโดยทุก ๆ คน) เขียนได้ว่า $\exists y \forall x \text{ Loves}(x, y)$
 - ในกรณีนี้ลำดับของ Quantifier มีความสำคัญ เช่น $\forall x (\exists y \text{ Loves}(x, y))$ และ $\exists y (\forall x \text{ Loves}(x, y))$
- กรณีที่ตัวบ่งปริมาณหลายตัวกับตัวแปรเดียวกัน
 - ในกรณีนี้ Innermost quantifier จะไม่อยู่ภายใต้ Quantifier ตัวอื่น
 - $\forall x (\text{Crown}(x) \vee (\exists x \text{ Brother}(\text{Richard}, x)))$
 - ประโยค $\exists x \text{ Brother}(\text{Richard}, x)$ เป็นประโยคเกี่ยวกับ Richard การใส่ $\forall x$ ไว้ข้างนอกจึงไม่มีผล ซึ่งมีค่าเท่ากับเขียนด้วยอีกตัวแปรคือ $\exists z \text{ Brother}(\text{Richard}, z)$
 - เราจึงมักเขียน Nested quantifier ด้วยตัวแปรที่มีชื่อที่ต่างกัน

Connections between $\forall$ and $\exists$

- หากสังเกตจะบอกว่า $\forall$ และ $\exists$ มีความสัมพันธ์กันผ่าน Negation
- เช่น การบอกว่า Everyone dislikes parsnips (ทุกคนไม่ชอบพาร์นิบส์) จะเหมือนกับการบอกว่า There does not exist someone who likes them (ไม่พบว่ามีบางคนชอบพาร์นิบส์)

- $\forall x \neg \text{Likes}(x, \text{Parsnips})$ จึงเท่ากับ (Equivalent) $\neg \exists x \text{ Likes}(x, \text{Parsnips})$

- $\forall x \text{ Likes}(x, \text{IceCream})$ เท่ากับ $\neg \exists x \neg \text{Likes}(x, \text{IceCream})$

- จึงมีกฎต่อไปนี้

$$\forall x \neg P \equiv \neg \exists x P$$

$$\neg \forall x P \equiv \exists x \neg P$$

$$\forall x P \equiv \neg \exists x \neg P$$

$$\exists x P \equiv \neg \forall x \neg P$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q)$$

- พบว่าเราอาจจะไม่ต้องการทั้ง $\forall$ และ $\exists$ คล้ายกรณี $\wedge$ หรือ $\vee$ แต่เราคงไว้เพื่อความง่ายในการอ่าน (readability)

Using First-order Logic

Using First-order³⁶ Logic

- จาก Slide ก่อนหน้าเราได้เห็นไวยากรณ์ของภาษาเชิงตรรกะมาแล้ว เราจะใช้ไวยากรณ์เหล่านี้ในการอธิบาย Domains ง่าย ๆ กัน
- ในการแทนองค์ความรู้ (Knowledge representation), **Domain** หมายถึงบางส่วนของโลกที่เราสนใจ

Assertions and queries in first-order logic³⁷

- ประโยคนั้นถูกเพิ่มเข้าไปใน Knowledge base (KB) ผ่าน TELL เหมือนใน Propositional logic
 - Sentences เรียกว่า Assertions เช่น เรามีการ Assert ประโยคต่อไปนี้ (ใน KB)
 - $\text{TELL}(\text{KB}, \text{King}(\text{John}))$.
 - $\text{TELL}(\text{KB}, \text{Person}(\text{Richard}))$.
 - $\text{TELL}(\text{KB}, \forall x \text{ King}(x) \Rightarrow \text{Person}(x))$.
 - เราสามารถ ASK ใน KB ได้ เช่น
 - $\text{ASK}(\text{KB}, \text{King}(\text{John}))$ คืนค่า True (Question ที่ถามใน ASK นั้นเรียกว่า **Queries** หรือ **Goals**)
 - เช่น มี Query $\text{ASK}(\text{KB}, \text{Person}(\text{John}))$ **Return True**
 - หรือจะใช้ Quantifier เป็น Query ก็ได้ เช่น $\text{ASK}(\text{KB}, \exists x \text{ Person}(x))$ **Return True**
 - แต่ประโยคตัวอย่างนี้ไม่ค่อยได้สาระอย่างที่เรายากได้คล้ายกับเราตอบคำถาม “Can you tell me the time?” ว่า “Yes.”
 - หากเราต้องการรู้ว่า x ตัวใดที่ทำให้ประโยคดังกล่าวเป็นจริงเราจะมีการใช้ฟังก์ชันหนึ่งคือ ASKVARs คือ
 - $\text{ASKVARs}(\text{KB}, \text{Person}(x))$ ซึ่งจะคืนค่ามาเป็นชุดของคำตอบในตัวอย่างนี้มี 2 คำตอบ $\{x/\text{John}\}$ และ $\{x/\text{Richard}\}$
 - คำตอบในลักษณะนี้เรียกว่า Substitution หรือ binding list

References

Stuart Russell and Peter Norvig. 2009. Artificial Intelligence: A Modern Approach (3rd. ed.). Prentice Hall Press, USA.